Simulación de Robots con Gazebo

Julio Garcia Ortiz

1. Entorno de trabajo 2
2. Descargamos la imagen de ROS con TurtleBot3 2
3. Creación de proyecto python 2
4. Comprobamos y lanzamos el contenedor ROS 2
5. Activamos ROS 3
6. Instalamos TurtleBot3 3
7. Configuramos el modelo del robot 3
8. Ejecución del programa 3
9. Error 3
10. Configurar gráficos en Windows 4
11. Reinicio del contenedor 4
12. Activación y lanzamiento 4
13. Relanzado de todo. 4
14. Copiamos el archivo Py al contenedor 5
15. Finalización 5

1. Entorno de trabajo

Antes de comenzar, verificamos que Docker está correctamente instalado en el sistema. En la terminal se ejecuta docker --version, confirmando la versión 29.2.1.

```
julio@JGO MINGW64 ~
$ docker --version
Docker version 29.2.1, build a5c7197
```

2. Descargamos la imagen de ROS con TurtleBot3

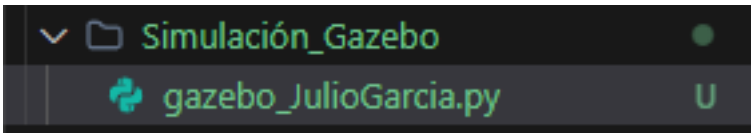
Se descarga la imagen oficial de ROS Noetic desde Docker Hub con el comando docker pull osrf/ros:noetic-desktop-full. La imagen pesa aproximadamente 6 GB e incluye todo el entorno necesario para trabajar con ROS y Gazebo.

```
julio@JGO MINGW64 ~
$ docker pull osrf/ros:noetic-desktop-full
noetic-desktop-full: Pulling from osrf/ros
4f15d5036206: Downloading [>
59cc10980325: Downloading [==>
```

3. Creación de proyecto python

Creamos la carpeta del proyecto y dentro el archivo de python que contendrá la lógica para que el robot pueda evitar chocar contra obstáculos.

```
julio@JGO MINGW64 ~/Desktop/Curso Especialización/IA-BigData/MIA-Modelos de Inte
ligencia Artificial/Robótica/Simulación_Gazebo (main)
$
```



4. Comprobamos y lanzamos el contenedor ROS

Con docker images verificamos que la imagen se descargó correctamente. Después lanzamos el contenedor en modo interactivo con *"docker run -it --name ros_gazebo osrf/ros:noetic-desktop-full bash"*, accediendo a su terminal como root.

```
julio@JGO MINGW64 ~/Desktop/Curso Especialización/IA-BigData/MIA-Modelos de Inte
ligencia Artificial/Robótica/Simulación_Gazebo (main)
$ docker images

IMAGE                          ID             DISK USAGE   CONTENT SIZE   EXTRA
osrf/ros:noetic-desktop-full   7dbfb9576d8e       6.01GB         1.16GB

julio@JGO MINGW64 ~/Desktop/Curso Especialización/IA-BigData/MIA-Modelos de Intelig
azebo (main)
$ docker run -it --name ros_gazebo osrf/ros:noetic-desktop-full bash
root@a4dd4c9b9b54:/#
```

5. Activamos ROS

Dentro del contenedor, ejecutamos ‘’*source /opt/ros/noetic/setup.bash*’’ para cargar el entorno de ROS y que estén disponibles todos sus comandos y librerías. Seguidamente ejecutamos ‘’*apt update*’’ para actualizar la lista de paquetes.

```
root@a4dd4c9b9b54:/# source /opt/ros/noetic/setup.bash
root@a4dd4c9b9b54:/# apt update
```

6. Instalamos TurtleBot3

Instalamos los paquetes necesarios para usar el robot TurtleBot3 dentro de Gazebo con el comando ‘’*apt install ros-noetic-turtlebot3 ros-noetic-turtlebot3-simulations -y*’’.

```
root@a4dd4c9b9b54:/# apt install ros-noetic-turtlebot3 ros-noetic-turtlebot3-simulations -y
Reading package lists... Done
```

7. Configuramos el modelo del robot

Definimos la variable de entorno que indica a ROS qué modelo de TurtleBot3 usaremos: ‘’*export TURTLEBOT3_MODEL=burger*’’. El modelo "burger" es el más básico y ligero de los tres disponibles.

```
root@a4dd4c9b9b54:/# export TURTLEBOT3_MODEL=burger
root@a4dd4c9b9b54:/#
```

8. Ejecución del programa

Lanzamos la simulación con ‘’*roslaunch turtlebot3_gazebo turtlebot3_world.launch*’’. ROS comienza a inicializarse y registrar logs, pero en este punto detectamos el problema del entorno gráfico.

```
root@a4dd4c9b9b54:/# roslaunch turtlebot3_gazebo turtlebot3_world.launch
... logging to /root/.ros/log/0c58edaa-2158-11f1-9c17-321654b91934/roslaunch-a4dd4c9b9b54-658.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.
```

9. Error

Ejecutó bien, pero dio error de que gazebo no podía usar la interfaz gráfica debido a que se está ejecutando en windows. Para arreglarlo detuve la ejecución para reiniciar con la configuración correcta.

```
julio@JGO MINGW64 ~/Desktop/Curso
azebo (main)
$ docker stop ros_gazebo
ros_gazebo

julio@JGO MINGW64 ~/Desktop/Curso
azebo (main)
$ docker rm ros_gazebo
ros_gazebo
```

10. Configurar gráficos en Windows

Configuramos la variable de entorno DISPLAY para permitir que las aplicaciones gráficas del contenedor se puedan mostrar en el sistema host “Windows”. Pero nos daremos cuenta de que esto no sirvió de na.

```
julio@JGO MINGW64 ~/Desktop/Curso Especializa
azebo (main)
$ setx DISPLAY host.docker.internal:0.0

CORRECTO: se guardó el valor especificado.
```

11. Reinicio del contenedor

Se vuelve a lanzar el contenedor de Docker, esta vez con la variable DISPLAY ya configurada en el sistema host, permitiendo teóricamente la visualización de la interfaz gráfica de Gazebo.

```
julio@JGO MINGW64 ~/Desktop/Curso E
azebo (main)
$ docker run -it --name ros_gazebo
root@4675c7c75c57:/#
```

12. Activación y lanzamiento

Al intentar lanzar de nuevo ROS y Gazebo, aparece un error indicando que el paquete turtlebot3_gazebo no se encuentra, porque este nuevo contenedor no tiene los paquetes instalados. Se repite el proceso: apt update y la instalación de los paquetes de TurtleBot3.

```
root@4675c7c75c57:/# source /opt/ros/noetic/setup.bash
root@4675c7c75c57:/# export TURTLEBOT3_MODEL=burger
root@4675c7c75c57:/# roslaunch turtlebot3_gazebo turtlebot3_world.launch
RLException: [turtlebot3_world.launch] is neither a launch file in package
 a launch file name
The traceback for the exception was written to the log file
root@4675c7c75c57:/#
```

```
root@4675c7c75c57:/# apt update
```

```
root@4675c7c75c57:/# apt install ros-noetic-turtlebot3 ros-noetic-turtlebot3-simulations -y
```

13. Relanzado de todo.

Instalamos de nuevo los paquetes en el nuevo contenedor y volvemos a relanzar el proceso.

```
root@4675c7c75c57:/# source /opt/ros/noetic/setup.bash
root@4675c7c75c57:/# export TURTLEBOT3_MODEL=burger
root@4675c7c75c57:/# roslaunch turtlebot3_gazebo turtlebot3_world.launch
```

14. Copiamos el archivo Py al contenedor

Desde el host Windows, copiamos el script de evasión de obstáculos al interior del contenedor con docker cp

```
julio@JGO MINGW64 ~/Desktop/Curso Especialización/IA-BigData/MI
azebo (main)
$ docker cp "C:\Users\julio\Desktop\Curso Especialización\IA-Bi
lación_Gazebo\gazebo_JulioGarcia.py" ros_gazebo:/root/
Successfully copied 2.56kB to ros_gazebo:/root/
```

15. Finalización

Dado que la interfaz gráfica de Gazebo no es accesible desde Windows sin un servidor X11 activo, se verifica el funcionamiento del sistema por terminal. Con "*docker exec -it ros_gazebo_fixed bash*" accedemos al contenedor en ejecución. Tras activar ROS, ejecutamos rostopic list y vemos los topics activos: */cmd_vel*, */rosout, /rosout_agg y /scan*. Esto confirma que el robot está publicando datos del sensor láser (*/scan*) y recibiendo comandos de movimiento (*/cmd_vel*), es decir, el programa de evasión de obstáculos está funcionando correctamente.

```
julio@JGO MINGW64 ~/Desktop/Curso Especialización/IA-BigData/MIA-
azebo (main)
$ docker exec -it ros_gazebo_fixed bash
root@864c49fbb050:/# source /opt/ros/noetic/setup.bash
root@864c49fbb050:/#
root@864c49fbb050:/# source /opt/ros/noetic/setup.bash
root@864c49fbb050:/# rostopic list
/cmd_vel
/rosout
/rosout_agg
/scan
root@864c49fbb050:/# rostopic echo /cmd_vel
```

Durante la realización de la práctica me encontré con varios problemas y terminé algo perdido por tener distintas terminales para ejecutar las pruebas, por este motivo puede que me haya saltado alguna captura o detalle por capturar.